Problem, hypothesis and controlled diagnosis

When 'healthy' is not enough

Vanshika Tiwari | Harshid Bhinde | Hritwik Ekade
94-879 Fundamentals of Operationalizing AI | Track B | October 10, 2026

Decision: ship B2's path repair and readiness check; use B1's append writer only for controlled, replayable runs until durability is addressed; in B3, alert and investigate rather than automatically retrain. These are distinct operational failures: falling behind, being unable to score, and serving changed inputs.

System and evidence boundary

Recorded ticks enter Kafka, become rolling features, persist to Parquet, and feed a model exposed through an API. Our presentation examined B1, B2 and B3. This brief supplements it with controlled measurements. B2's September 25 captures survive. Original B1 logs and the original B3 model/script do not; their numerical results are not treated as verified final evidence.

B1: hypothesis and falsification

The initial hypothesis was that more partitions and consumers would remove growing lag; unchanged lag would falsify it. The presentation reported that falsification. The producer's constant BTC-USD key explains why the added consumers could remain idle: one key is routed to one partition. We did not repeat the historical partition comparison and do not claim to recover its measurements.

The replacement hypothesis is that reading and rewriting all prior rows every 25 features makes storage cost grow with accumulated data. If save time stays flat as existing row count increases, or the append change fails under equivalent load, this diagnosis weakens. Rewriting successively larger files entails quadratic cumulative row-serialization work; a throughput decline over time alone cannot establish the cause.

Controlled save test

Seven randomized trials per writer and file size; identical 25-row additions, schema, host and storage. Seed-file creation is outside the timer. Existing rows vary independently of elapsed run time. Append footer closure is outside its save timer: the operations have different durability guarantees.

Rows already stored	Rewrite median (ms)	Append median (ms)
0	3.94	3.72
5,000	22.68	1.04
20,000	26.93	1.01
80,000	46.44	1.15

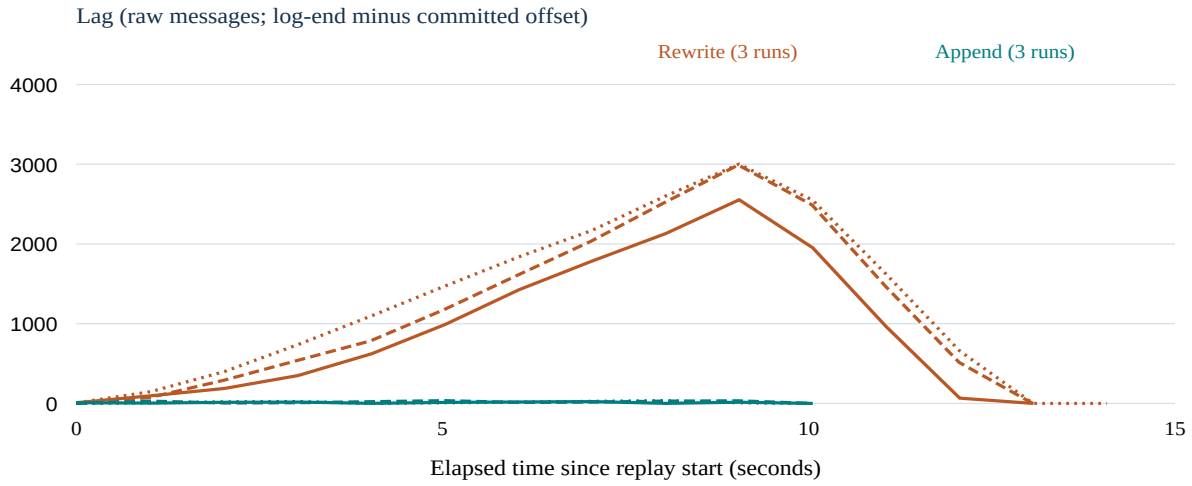
Rewrite cost rises with stored rows while open-writer append cost is nearly flat after initialization. This supports the code-based mechanism, not a fitted asymptotic timing law. Measurements are warm-cache and do not include fsync or power-failure testing.

Evidence: b1_save_trials.csv (56 trials); b1_save_summary.csv; supplied baseline _append_parquet and submitted _ParquetAppender.

B1: measured improvement, bounded recommendation

Equivalent finite replay, repeated three times

Each run starts with a unique empty topic and group, one partition/consumer, and a fresh output file. The same 14,076 raw messages yield 11,754 features; non-ticker messages do not produce rows. Input timestamps have zero backward steps. Replay is finite, never looped. Run order alternates; final timing runs were collected without concurrent benchmark/bootstrap work.



Measure (3 runs each)	Rewrite	Append
Achieved raw arrivals/s	1,499.8-1,499.9	1,499.7-1,499.8
Peak sampled lag (messages)	2,554-3,001	23-35
Observed completion (seconds)	13.09-14.11	10.12-10.13
Features / observed second	833.1-898.1	1,160.4-1,161.4
Final sampled lag	0 in all runs	0 in all runs
Exact feature-row equality	11,754 / 11,754	11,754 / 11,754

Achieved rate uses acknowledged delivery count divided by producer elapsed time, including final flush. Lag is sampled about every second; completion times include observation delay. Every output column and row matches a direct single-pass calculation exactly. This supports the bounded finite-load improvement, not a multi-hour capacity claim.

Hard kill reverses an unconditional ship recommendation

A Linux SIGKILL after 40 completed 25-row writes leaves 1,000/1,000 rows readable for rewrite and 0/1,000 normally readable for append. The open file lacks its footer: all 1,000 rows are unavailable to standard PyArrow reads, not just the last row group. This writer-only test performs no Kafka commits; it does not measure end-to-end recovery loss. Production code commits after appends, so unreadable committed output is a real risk.

Keep the SIGTERM/SIGINT close handler, but do not equate graceful closure with crash durability. The old in-place rewrite is also vulnerable if killed during a write. Use append only with retained replayable inputs and fresh output paths; durable closed segments and offset/data recovery must be validated before production.

Evidence: b1_stream_summary.csv; b1_stream_telemetry.csv and six logs; b1_sigkill.json. Original 73,607 lag and 3,000/s achieved-rate claims are withdrawn: looping or pre-existing backlog cannot be distinguished without the missing logs.

B2: make deployment health agree with scoring

Diagnosis and controlled causal test

The supplied B2 configuration points MODEL_PATH to /app/models/artifact/model.pkl, while the mounted directory is artifacts. The original process catches a missing file and stays alive. Docker probes /health and sees HTTP 200 even when /score returns 503. This is both a configuration defect and a detection defect.

Hypothesis: changing only MODEL_PATH, keeping image, mount and scoring payload constant, restores scoring. A continued 503 would reject that explanation. Historical captures show the configured path absent, the mounted file present, and the path-only override restoring HTTP 200 and the baseline score. Three restarts with the wrong path all reproduced the fault; restarting is not the repair.

Intervention

Repair the model path. Keep /health as a cheap liveness response. Add /ready: require a loaded model, run a canary through the same predict function as /score, and require a finite probability in [0,1]. Return HTTP 503 with a diagnostic reason on failure. Point Docker's healthcheck to /ready. Model-load exceptions are surfaced as unready instead of silently looking usable.

Controlled condition	Docker	/health	/ready	/score
Original + bad path	healthy	200	absent	503
Repaired + hardened	healthy	200	200	200
Hardened + bad path	unhealthy	200	503	503

The historical probe timeline was sampled every 10 seconds: starting at 0-40 seconds and unhealthy at 50 seconds after five failed probes. Liveness remained 200 after startup. A corrupt artifact made the original process exit with code 1; the hardened service stayed up and exposed UnpicklingError through readiness.

Cost and verification

Historical host-side latency: 300 sequential requests per endpoint after 20 warm-ups. /health p50 3.50 ms, p95 5.51 ms; /ready p50 4.65 ms, p95 6.03 ms. Median difference is 1.15 ms. These are local, sequential probes, not a concurrency benchmark; raw individual timings are unavailable.

The final repository includes the previously local hardened scorer and repaired compose files. Supplemental Flask test-client checks with the newly generated model confirm health/ready/score = 200/200/200 for a valid artifact and 200/503/503 for missing and corrupt artifacts. These are application-level regression checks, not a new Docker timing run.

Deployment recommendation and limits

Ship both changes. In Compose, unhealthy is a status label: it does not remove traffic or restart the container. A stale canary could mark all replicas unready, and a runnable but wrong model can pass. Monitor ready replicas, caller 5xx, load errors and time-to-ready. Test traffic gating and failed rollouts separately before claiming Kubernetes reliability.

Evidence: evidence/b2_historical/01-08 captures, especially 03_diagnosis, 05_fault_injected_hardened_code and 07_probe_cost; supplemental/results.json, b2 section.

B3: material drift, limited model-quality evidence

Hypothesis partly rejected

The initial hypothesis linked wider spreads to detectable spread_normalized drift and a consequential model effect. Its stated rejection rule included drift with an unaffected model. Detection is supported; demonstrated degradation is not. We therefore partly reject the original combined hypothesis. An investigation is still warranted because an input contract changed substantially and this small synthetic test cannot establish safety.

Resolve the spread measurement

Both input files contain 782 raw messages and 653 feature rows, computed with the same featurizer. Mean normalized spread rises 3.9792x; mean raw spread rises 3.9792x. The first tick changes from 0.01 to 0.05 (5x), but per-tick ratios vary approximately 3x-5x. Thus 5x is an example, not the aggregate change.

Feature	KS statistic	Interpretation
spread_normalized	0.9525	Large distribution shift
midprice_returns	0.0046	Small rounding perturbation
order_book_imbalance	0.0000	Unchanged
trade_intensity	0.0000	Unchanged

Direct SciPy KS calculations independently corroborate the presentation's feature-level finding. Spread p-value is below numerical precision; other p-values are 1. Serial dependence limits literal significance interpretation. One of four features shifted; a 50% dataset-level rule can miss this consequential single-feature change.

New, reproducible model-sensitivity evaluation

The original slide model is unavailable. We fit a new balanced logistic regression and scaler on the first 371 rows, purging 20 rows before the 60% training boundary. Test indices 522-632 provide 111 paired rows after removing the unlabeled 20-row tail. Proxy labels use forward 20-row return volatility and a reference-wide 90th-percentile cutoff, held fixed across the synthetic perturbation; 19 test labels are positive.

This label definition is retrospective, not prospective validation. A cutoff estimated only from training gives zero positive test cases. Trade prices are identical, but bid/ask rounding slightly perturbs midprice returns (maximum absolute change 9.85e-8). No model or scaler fitting uses test rows.

Average precision (AP)	Estimate	95% block-bootstrap interval
Reference	0.208	[0.031, 0.502]
Current	0.254	[0.037, 0.576]
Current minus reference	0.046	[0.000, 0.163]

AP is sklearn average_precision_score, commonly called PR-AUC here; it is not trapezoidal PR area. Paired circular moving-block bootstrap: 3,000 attempts, block length 20, 2,732 valid two-class draws. The delta interval includes zero. These are not intervals for the slide's 0.208/0.303 result and do not justify calling that result noise.

Evidence: results.json; b3_drift.csv; b3_paired_scores.csv; b3_bootstrap.csv. Block length 10 sensitivity: delta 95% interval [0.000, 0.171], 2,902 valid draws. Intervals omit model-training uncertainty.

Operational actions and tradeoffs

Threshold sensitivity changes the interpretation

Threshold	Alerts: ref / current	Precision: ref / current	Recall: ref / current
0.3	108/111 / 107/111	0.157 / 0.150	0.895 / 0.842
0.5	100/111 / 98/111	0.170 / 0.153	0.895 / 0.789
0.7	72/111 / 70/111	0.181 / 0.186	0.684 / 0.684
0.9	0/111 / 0/111	undefined / undefined	0.000 / 0.000

At 0.5, 90.1% of reference and 88.3% of current rows are flagged: the alert rate is saturated and weak as a standalone quality signal. Recall falls from 0.895 to 0.789 even though AP rises, so 'the model is unaffected' is too strong. At 0.7, rates fall to 64.9% and 63.1%, with equal 0.684 recall. At 0.9 no positives are predicted; precision is undefined. Thresholds are illustrative, not selected using this tiny test tail.

Mean absolute score change is 0.0153. Metric behavior is threshold-dependent and intervals are wide. We observe sensitivity to the perturbation, but cannot establish consistent degradation, equivalence, or improvement in production. Keep serving while investigating; do not automatically retrain from a drift p-value or AP point estimate.

Proposed monitoring and response policy

Signal	Trigger / response	Owner
Infrastructure / readiness	Zero ready replicas or scoring 5xx: page; inspect path/artifact, restore known-good config.	Service operator
B1 lag + freshness	Sustained lag growth or stale output: inspect save cost; reduce intake or replay safely.	Pipeline owner
Feature drift	Proposed: KS effect >0.2 AND spread mean ratio >1.5; investigate feed/units and market context.	Model + data owner
Model quality	Collect delayed labels; compare precision/recall and AP with uncertainty and operating cost.	Model owner

The drift trigger is an operational proposal, not a pre-registered or false-alarm-calibrated threshold. Add window minimums, persistence and deduplication before alerting continuously. Infrastructure health, data validity and model quality must remain separate signals. A per-feature alert is appropriate even if the dataset-wide fraction is below its threshold.

Deployment gates

B1: preserve source data and use a fresh output path for every run. The current writer can truncate a pre-existing path on restart. Before production, require closed durable segments, an explicit offset/data recovery protocol and kill-during-write/restart tests. Do not infer exactly-once durability from a committed Kafka offset.

B2: release path repair plus readiness after canary-schema review. Provide operator rollback because repeated restart cannot repair a missing model. Validate model identity separately; readiness proves execution, not correctness. Pin tested image digests before production; the starter still uses a mutable Kafka latest tag.

B3: investigate source/units, retain both windows and collect labels from an actual wide-spread regime. Retraining requires sustained, consequential labeled degradation and validation of the proposed replacement. Sensitive alerts incur operator load; suppressing a single-feature shift risks overlooking a broken input contract.

Evidence: b3_thresholds.csv and b3_paired_scores.csv. Proposed operational triggers are recommendations, not tested SLAs or production guarantees.

AI supervision, reproducibility and remaining uncertainty

Consequential AI decision 1: replace unverifiable evidence

AI proposed replacing the unverifiable B1 slide measurements with new controlled experiments, rather than treating the configured 3,000/s rate and 73,607 lag as established. Disposition: accepted by the submitting team member during final preparation. Reason: 73,607 lag cannot arise from a clean one-pass 14,076-message workload, and the original run mode/offset history cannot be recovered.

Verification: new unique Kafka topics/groups; 14,076 acknowledged arrivals per run; measured arrival rates; zero backward timestamps; six before/after runs; every one of 11,754 output rows exactly equal to the direct feature reference. A separate randomized file-size experiment isolates storage cost. Missing historical logs remain missing; the new results do not retroactively validate the old charts.

Consequential AI decision 2: restrict the B1 recommendation

AI proposed modifying 'ship append with a shutdown handler' to conditional use pending durable storage and recovery. Disposition: accepted by the submitting team member after reviewing the SIGKILL result. Reason: a graceful close handler cannot run under SIGKILL, and the open file's missing footer makes the entire file unreadable through the normal reader.

Verification: the harness completed 40 batches (1,000 rows), then delivered actual Linux SIGKILL. Rewrite retained 1,000 readable rows; append retained zero. The brief distinguishes this writer-level result from unmeasured Kafka recovery loss and withdraws the old blanket 'crash-safe every flush' claim. The team retains responsibility for the deployment decision and interpretation.

How the working artifact can be checked

From the repository root, install experiments/requirements.txt under Python 3.12 and run experiments/validate_submission.py. Start the supplied Kafka broker and run experiments/stream_comparison.py --rate 1500. On Linux, run validate_submission.py --crash for SIGKILL. The root README includes Docker commands, B2 fault injection and repair, environment details and expected outputs.

Scripts, input hashes, raw trial CSVs, lag telemetry, worker logs, paired scores and bootstrap samples are included. B2 captures remain dated September 25; supplemental evidence is dated October 10. The original baseline is retained in the supplied challenge package and initial Git commit. The PDF and editable text summarize evidence; they are not substitutes for the raw outputs.

Most important remaining uncertainty

Can the faster B1 pipeline preserve or recover all committed output after a hard failure and restart? Until a durable-write/offset protocol is implemented and tested, the observed throughput benefit does not establish production reliability. Separately, B3 needs representative delayed labels: synthetic paired inputs and retrospective proxy labels cannot tell us whether the system remains useful in a real wide-spread regime.

Sources and supporting material

Assignment: 94-879 Final Team Project Track B - Technical Challenges, Final Deliverables (six-page brief limit). Team presentation: When Healthy Isn't Working, October presentation; numerical discrepancies are corrected here. Instructor's post-presentation feedback motivates the supplemental tests. Repository: vanshika1631/OpsAI_TrackB, initial commit eae698f. Internal evidence index: evidence/README.md. Feedback closure: submission/feedback_response.md.

AI decisions recorded in the final-preparation conversation: user explicitly accepted both recommendations. No unavailable historical AI advice or verification is invented.